

How I developed the program

I first started of using the templates provided. There I changed the old and deprecated API's for more modern ones, before getting right of the keyboard actions entirely. I first got a basic render working of the Mandelbrot. Afterwards, I split the version up into the different folders "gui" for rendering and display, "util" for the different color palettes, "logic" for the fractal math and the rendering logic as well as a general "Main.java" file, so that the project can be run as requested.

I used javax swing library for display as well as implementing user input. The familiar java util concurrent library was used for anything to do with parallelization and threads management.

The user can choose the amount of threads in the GUI dropdown, (1, 2, 4, 8 or 16). The user can also switch between the color palettes, and either in- or decrease the amount of iterations done for rendering the fractal. There is also a "Reset Zoom" button for resetting to the starting view of the fractal renderer. To resize the fractal renderer, the user can simply press the "Reset Zoom" button after resizing the window as usual.

How I parallelized the program

The primary parallelization mechanism used in my program is a thread pool. The idea here is quite similar to what we looked at in class, since the pixels themselves can be calculated completely independently from each other. This allows for a nice application of thread pools, similar to what we did with the matrix operations. Upon calling the draw function of the renderer, a thread pool with size matching the user requested amounts of threads is created. We then submit a task for each of the threads created, and divide the pixels among the tasks in horizontal rows. The way this works is that each thread calculates the pixels of the rows which are a multiple of its thread id, meaning that for 4 threads, thread 0 handles the rows 0, 4, 8, 12, etc. while thread 1 deals with rows 1, 5, 9, etc.

Why this is correct

I use "y += threadCount" for assigning the rows to the threads, meaning that no two threads will ever work on the same row. Because of this, we run no risk of overwriting the same pixels on accident. If the user zooms or the render becomes invalid in some other way, the program uses two volatile fields "dirty" and "cancelled". Dirty makes sure that the threads only run if something has changed, and a new render has to be done. Cancelled allows for clean interruption of current renders taking place.

Also, all of the user input methods which could potentially cause interrupts and invalid states are marked with the keyword synchronized, making sure that nothing goes wrong there. I think there is honestly not much room for parallelism to go wrong in my program.

How to use the program

To use the program, either compile and run it in a terminal using:

```
PS C:\Users\Dom\Documents\Bachelor_Comp_Sci\Semester\2026_FS\MCP> javac -d  
bin .\miniproject\mandelbrot_multithreader\Main.java
```

```
PS C:\Users\Dom\Documents\Bachelor_Comp_Sci\Semester\2026_FS\MCP> java -cp  
bin miniproject.mandelbrot_multithreader.Main
```

Or by running it through some IDE supported way of compiling and executing the Main.java file.

The different parameters can only be set in the GUI, through the very intuitive and labeled buttons and drop downs. As already mentioned, it is possible to change the amount of threads used, the amount of iterations done in the fractal calculation, switching the color palette and resetting the zoom into the fractal. To actually zoom into the fractal, simply hold down the mouse and drag appearing rectangle to the shape that you want to zoom into.

The speedup obtained by my program

On average, I have obtained almost a twofold increase in speed when going from 1 to 2 threads, and over a three times faster render time when going from 1 to 4 threads. Below are two examples of this speedup, the first done with a zoomed in view and 270 iterations, the second one with almost 1'000 iterations.

270 iterations	1 thread	2 threads	4 threads
Render time	420-510ms	219-270ms	140-156ms
Speedup		~83%	~314%

991 iterations	1 thread	2 threads	4 threads
Render time	1600-1800ms	980-1080ms	538-618ms
Speedup		~73%	~309%